

1. somar_naturais(n)

COM CAUDA

```
def somar_naturais(n), do: somar_naturais_cauda(n, 0)
defp somar_naturais_cauda(0, acc), do: acc
defp somar_naturais_cauda(n, acc), do: somar_naturais_cauda(n - 1, acc + n)
```

SEM CAUDA

```
def somar_naturais(0), do: 0
def somar_naturais(n), do: n + somar_naturais(n - 1)
```

2. fatorial(n)

COM CAUDA

```
def fatorial(n), do: fatorial_cauda(n, 1)
defp fatorial_cauda(1, acc), do: acc
defp fatorial_cauda(n, acc), do: fatorial_cauda(n - 1, acc * n)
```

SEM CAUDA

```
def fatorial(0), do: 1
def fatorial(n), do: n * fatorial(n - 1)
```

3. fibonacci(n)

COM CAUDA

```
def fibonacci(n), do: fibonacci_cauda(n, 0, 1)
defp fibonacci_cauda(1, atual, _proximo), do: atual
defp fibonacci_cauda(n, atual, proximo), do: fibonacci_cauda(n - 1, proximo, atual + proximo)
```

SEM CAUDA

```
def fibonacci(0), do: 0
def fibonacci(1), do: 1
def fibonacci(n), do: fibonacci(n - 1) + fibonacci(n - 2)
```

4. menor_de_dois(a, b)

COM CAUDA

```
def menor_de_dois(a, b), do: menor_cauda(a, b, 0)
defp menor_cauda(0, _b, acc), do: acc
defp menor_cauda(_a, 0, acc), do: acc
defp menor_cauda(a, b, acc), do: menor_cauda(a - 1, b - 1, acc + 1)
```

SEM CAUDA

```
def menor_de_dois(a, b) when a < b, do: a
def menor_de_dois(_a, b), do: b
```

5. produtorio_impares(n)

COM CAUDA

```
def produtorio_impares(n), do: produtorio_impares_cauda(n, 1)
defp produtorio_impares_cauda(0, acc), do: acc
defp produtorio_impares_cauda(n, acc) do
  impar_atual = (2 * n) - 1
  produtorio_impares_cauda(n - 1, acc * impar_atual)
end
```

end

SEM CAUDA

```
def produtorio_impares(0), do: 1
def produtorio_impares(n), do: (2 * n - 1) * produtorio_impares(n - 1)
```

#6. Somatorio dos quadrados

COM CAUDA

```
def somatorio_quadrados(n), do: somatorio_quadrados_cauda(n, 0)
defp somatorio_quadrados_cauda(0, acumulador), do: acumulador
defp somatorio_quadrados_cauda(n, acumulador) do
  somatorio_quadrados_cauda(n - 1, (n * n) + acumulador)
end
```

SEM CAUDA

```
def somatorio_quadrados(0), do: 0
def somatorio_quadrados(n), do: (n * n) + somatorio_quadrados(n - 1)
```

7. Soma dos dígitos

COM CAUDA

```
def soma_digitos(n), do: soma_digitos_cauda(n, 0)
defp soma_digitos_cauda(0, acc), do: acc
defp soma_digitos_cauda(n, acc), do: soma_digitos_cauda(div(n, 10), acc + rem(n, 10))
```

SEM CAUDA

```
def soma_digitos(n) when n < 10, do: n
def soma_digitos(n), do: rem(n, 10) + soma_digitos(div(n, 10))
```

8. MDC

COM CAUDA

```
def mdc(a, b), do: mdc_cauda(a, b)
defp mdc_cauda(a, 0), do: a
defp mdc_cauda(a, b), do: mdc_cauda(b, rem(a, b))
```

SEM CAUDA

```
def mdc(a, 0), do: a
def mdc(a, b), do: mdc(b, rem(a, b))
```

9. Contagem regressiva

COM CAUDA

```
def contagem_regressiva(n), do: contagem_regressiva_cauda(n, [])
defp contagem_regressiva_cauda(0, acc), do: acc
# Usamos acc ++ [n] para manter a ordem n...1 de forma recursiva de cauda
defp contagem_regressiva_cauda(n, acc), do: contagem_regressiva_cauda(n - 1, acc ++ [n])
```

SEM CAUDA

```
def contagem_regressiva(0), do: []
def contagem_regressiva(n), do: [n | contagem_regressiva(n - 1)]
```

10. Soma harmônica

COM CAUDA

```
def soma_fracoas(n), do: soma_fracoas_cauda(n, 0.0)
defp soma_fracoas_cauda(0, acc), do: acc
```

```
defp soma_fracoes_cauda(n, acc), do: soma_fracoes_cauda(n - 1, acc + (1 / n))
end
```

SEM CAUDA

```
def soma_fracoes(1), do: 1.0
def soma_fracoes(n), do: (1 / n) + soma_fracoes(n - 1)
```

11. Tamanho

COM CAUDA

```
def tamanho(lista), do: tamanho_cauda(lista, 0)
defp tamanho_cauda([], acc), do: acc
defp tamanho_cauda([_h | t], acc), do: tamanho_cauda(t, acc + 1)
```

SEM CAUDA

```
def tamanho([]), do: 0
def tamanho([_h | t]), do: 1 + tamanho(t)
```

12. Somar elementos

COM CAUDA

```
def somar_elementos(lista), do: somar_elementos_cauda(lista, 0)
defp somar_elementos_cauda([], acc), do: acc
defp somar_elementos_cauda([h | t], acc), do: somar_elementos_cauda(t, acc + h)
```

SEM CAUDA

```
def somar_elementos([]), do: 0
def somar_elementos([h | t]), do: h + somar_elementos(t)
```

13. Multiplicar elementos

COM CAUDA

```
def multiplicar_elementos(lista), do: multiplicar_elementos_cauda(lista, 1)
```

```
defp multiplicar_elementos_cauda([], acc), do: acc
defp multiplicar_elementos_cauda([h | t], acc), do: multiplicar_elementos_cauda(t, acc * h)
```

SEM CAUDA

```
def multiplicar_elementos([], do: 1
def multiplicar_elementos([h | t]), do: h * multiplicar_elementos(t)
```

14. Pertence?

COM CAUDA

```
def pertence?(elemento, lista), do: pertence_cauda(elemento, lista)
defp pertence_cauda(_elemento, []), do: false
defp pertence_cauda(elemento, [elemento | _t]), do: true
defp pertence_cauda(elemento, [_h | t]), do: pertence_cauda(elemento, t)
```

SEM CAUDA

```
def pertence?(_elemento, []), do: false
def pertence?(elemento, [h | t]), do: h == elemento or pertence?(elemento, t)
```

15. Inverter

COM CAUDA

```
def inverter(lista), do: inverter_cauda(lista, [])
defp inverter_cauda([], acc), do: acc
defp inverter_cauda([h | t], acc), do: inverter_cauda(t, [h | acc])
```

SEM CAUDA

```
def inverter([]), do: []
def inverter([h | t]), do: inverter(t) ++ [h]
```

16. Maior

COM CAUDA

```
def maior([h | t]), do: maior_cauda(t, h)
defp maior_cauda([], acc), do: acc
defp maior_cauda([h | t], acc) do
  novo_maior = if h > acc, do: h, else: acc
  maior_cauda(t, novo_maior)
```

SEM CAUDA

```
def maior([h]), do: h
def maior([h | t]) do
  m = maior(t)
  if h > m, do: h, else: m
end
```

17. maiores_que(elemento, lista)

COM CAUDA

```
def maiores_que(e, lista), do: maiores_que_c(e, lista, [])
defp maiores_que_c(_e, [], acc), do: Enum.reverse(acc)
defp maiores_que_c(e, [h | t], acc) do
  novo_acc = if h > e, do: [h | acc], else: acc
  maiores_que_c(e, t, novo_acc)
end
```

SEM CAUDA

```
def _que(_e, []), do: []
def maiores_que(e, [h | t]) do
  if h > e, do: [h | maiores_que(e, t)], else: maiores_que(e, t)
end
```

18. maiores_que?(elemento, lista)

COM CAUDA

```
def maiores_que?(e, lista), do: mq_bool_c(e, lista)
defp mq_bool_c(_e, []), do: true
defp mq_bool_c(e, [h | t]) do
  if h > e, do: mq_bool_c(e, t), else: false
end
```

SEM CAUDA

```
def _que(_e, []), do: []
def maiores_que(e, [h | t]) do
  if h > e, do: [h | maiores_que(e, t)], else: maiores_que(e, t)
end
```

19. contar_ocorrencias(elemento, lista)

COM CAUDA

```
def contar_ocorrencias(e, lista), do: contar_c(e, lista, 0)
```

```

defp contar_c(_e, [], acc), do: acc
defp contar_c(e, [h | t], acc) do
  novo_acc = if h == e, do: acc + 1, else: acc
  contar_c(e, t, novo_acc)
end

```

SEM CAUDA

```

def contar_ocorrencias(_e, []), do: 0
def contar_ocorrencias(e, [h | t]) do
  if h == e, do: 1 + contar_ocorrencias(e, t), else: contar_ocorrencias(e, t)
end

```

20. remover(elemento, lista)

COM CAUDA

```

def remover(e, lista), do: remover_c(e, lista, [])
defp remover_c(_e, [], acc), do: Enum.reverse(acc)
defp remover_c(e, [e | t], acc), do: Enum.reverse(acc) ++ t
defp remover_c(e, [h | t], acc), do: remover_c(e, t, [h | acc])
end

```

SEM CAUDA

```

def remover(_e, []), do: []
def remover(e, [e | t]), do: t
def remover(e, [h | t]), do: [h | remover(e, t)]
end

```

#21. eh_conjunto?

COM CAUDA

```

def eh_conjunto?(lista), do: eh_conjunto_cauda(lista)
defp eh_conjunto_cauda([]), do: true
defp eh_conjunto_cauda([h | t]) do
  if pertence?(h, t), do: false, else: eh_conjunto_cauda(t)
end

```

SEM CAUDA

```

def eh_conjunto?([]), do: true
def eh_conjunto?([h | t]) do
  if pertence?(h, t), do: false, else: eh_conjunto?(t)
end

```

22. uniao (Começamos o acumulador com o conjunto B e adicionamos o que falta de A)

COM CAUDA

```
def uniao(conjunto_A, conjunto_B), do: uniao_cauda(conjunto_A, conjunto_B)
defp uniao_cauda([], acc), do: acc
defp uniao_cauda([h | t], acc) do
  novo_acc = if pertence?(h, acc), do: acc, else: [h | acc]
  uniao_cauda(t, novo_acc)
end
```

SEM CAUDA

```
def uniao([], conjunto_B), do: conjunto_B
def uniao([h | t], conjunto_B) do
  if pertence?(h, conjunto_B) do
    uniao(t, conjunto_B)
  else
    [h | uniao(t, conjunto_B)]
  end
end
```

23. Intersecao

COM CAUDA

```
def intersecao(conjunto_A, conjunto_B), do: intersecao_cauda(conjunto_A, conjunto_B, [])
defp intersecao_cauda([], _cB, acc), do: acc
defp intersecao_cauda([h | t], conjunto_B, acc) do
  novo_acc = if pertence?(h, conjunto_B), do: [h | acc], else: acc
  intersecao_cauda(t, conjunto_B, novo_acc)
end
```

SEM CAUDA

```
def intersecao([], _conjunto_B), do: []
def intersecao([h | t], conjunto_B) do
  if pertence?(h, conjunto_B) do
    [h | intersecao(t, conjunto_B)]
  else
    intersecao(t, conjunto_B)
  end
end
```

24. Pertence?

COM CAUDA

```
def pertence?(elemento, conjunto), do: pertence_cauda(elemento, conjunto)
```

```

defp pertence_cauda(_elemento, []), do: false
defp pertence_cauda(elemento, [elemento | _t]), do: true
defp pertence_cauda(elemento, [_h | t]), do: pertence_cauda(elemento, t)

```

SEM CAUDA

```

def pertence?(_elemento, []), do: false
def pertence?(elemento, [h | t]), do: h == elemento or pertence?(elemento, t)

```

25. Subconjunto?

COM CAUDA

```

def subconjunto?(conjunto_A, conjunto_B), do: subconjunto_cauda(conjunto_A,
conjunto_B)
defp subconjunto_cauda([], _cB), do: true
defp subconjunto_cauda([h | t], conjunto_B) do
  if pertence?(h, conjunto_B) do
    subconjunto_cauda(t, conjunto_B)
  else
    false
  end
end
end

```

SEM CAUDA

```

def subconjunto?([], _conjunto_B), do: true
def subconjunto?([h | t], conjunto_B) do
  pertence?(h, conjunto_B) and subconjunto?(t, conjunto_B)
end

```

26. Superconjunto?

COM CAUDA

```

def superconjunto?(conjunto_A, conjunto_B), do: subconjunto?(conjunto_B, conjunto_A)
end

```

SEM CAUDA

```

def superconjunto?(conjunto_A, conjunto_B) do
  subconjunto?(conjunto_B, conjunto_A)
end
end

```